- In code, by obtaining an instance of your `AppDbContext` from the DI container and calling `context.Database.Migrate()`
- By generating a migration bundle application (see http://mng.bz/jPyr)

Which method is best for you depends on how you designed your application, how you'll update your production database, and what your personal preference is. I'll use the .NET tool for now, but I discuss some of these considerations in section 12.5. You can apply migrations to a database by running

```
dotnet ef database update
```

from the project folder of your application. I won't go into the details on how this command works, but it performs four steps:

1. Builds your application
2. Loads the services configured in your app's Program.cs, including `AppDbContext`
3. Checks whether the database in the `AppDbContext` connection string exists and if not, creates it
4. Updates the database by applying any unapplied migrations

If everything is configured correctly, as in section 12.2, running this command sets you up with a shiny new database like the one shown in figure 12.7.